# Chapter 10: Reusable Item Embeddings

## 10.1 Introduction: The Case for Reusability

### The Proliferation Problem

In the preceding chapters, we built a multi-stage recommendation pipeline: retrieval narrows millions of candidates to hundreds (Chapter 5), ranking scores them (Chapters 6–7), and value functions translate predicted engagement into business outcomes (Chapter 8). In a real enterprise, however, this pipeline does not exist in isolation. A typical organization runs many recommendation surfaces simultaneously such as a homepage carousel, a "related items" widget, an email notification system, and perhaps a parallel portfolio of various in-house ad models. It is not uncommon for each of these surfaces to be owned by a different team, each building and maintaining its own models, its own feature pipelines, and its own training infrastructure.

Over time, this proliferation of bespoke models becomes a serious organizational burden. Different groups build separate recommenders for different surfaces, each optimized for its narrow task, and the enterprise ends up maintaining a growing collection of algorithms and pipelines that have significant overlap in functionality. The data preprocessing is similar. The feature engineering is duplicated. The evaluation metrics are computed independently. Yet each pipeline is different enough that an improvement invented by one team cannot easily be transferred to another's model. Sculley et al.'s influential paper on technical debt in machine learning systems warns that having many specialized pipelines with similar steps is a recipe for long-term inefficiency – a hidden technical debt that compounds silently until it becomes a crisis.

Netflix encountered exactly this problem. They had distinct models for notifications (user→item), for "more like this" suggestions (item→item), for search ranking (query→item), and several others. Each model was trained separately despite drawing from common data sources. As the Netflix engineering team described it, this approach created systems management overhead and made it difficult to transfer innovations from one model to another. Their solution was to consolidate into a unified model that serves many use cases, sharing knowledge between tasks and reducing the number of separate systems to maintain. Uber's journey with restaurant recommendations on Uber Eats followed a similar arc: they replaced thousands of city-specific models with a single global two-tower embedding model, dramatically reducing complexity while maintaining performance.

### Recognizing the Reusability Hierarchy

As engineering leaders begin to recognize the unsustainability of the bespoke model approach, they naturally ask: what exactly can be shared across recommendation use cases?

The answer is more than most people initially expect. We can think of a hierarchy of reusability, from the most abstract knowledge down to concrete trained artifacts.

At the most foundational level, teams can reuse ideas and design patterns. The concept of collaborative filtering, the two-stage retrieval-then-ranking architecture, the principle that implicit feedback often outperforms explicit ratings, etc. These are proven **approaches** that apply across virtually every recommendation domain. Rather than formulating a novel technique for each new surface, engineers can apply or adapt patterns that have worked at companies like YouTube, Pinterest, and Alibaba, customizing only the parts that genuinely require domain-specific insight.

One level up, organizations can reuse data and feature pipelines. Many recommendation tasks within a company draw from the same pools of interaction data such as clicks, views, purchases, search queries. Building a shared feature store where common features are computed once and made available to all recommender teams eliminates duplicated ETL & feature engineering work and ensures consistency. (We covered some of the Feature Store considerations for feature creation earlier in Chapter 4). When every model uses the same definition of "user engagement score" or "item popularity count," results become comparable and debugging becomes tractable.

Further up in the hierarchy, teams can reuse code through common libraries and frameworks. The open-source ecosystem provides a strong starting point: libraries like TensorFlow Recommenders, RecBole, and LightFM implement popular Recsys algorithms under consistent APIs, allowing engineers to swap models with a configuration change rather than a rewrite. Internal libraries that standardize training loops, evaluation metrics, and data loading further reduce the per-project cost of standing up a new recommender.

In the author's experience, the highest-ROI investment that organizations can reuse are learned representations, particularly embeddings. This is where the leverage is greatest. A well-trained item embedding can be consumed directly by a retrieval model for candidate generation (Chapter 5), fed as input features into ranking models (Chapters 6–8), used for ad targeting (Chapter 9), and serve as the foundation for user embeddings (Chapter 11). When an organization invests in high-quality item representations, every downstream model benefits. Conversely, when item embeddings are poor, such as when a headphone item and a garden hose are equidistant from a laptop product, no amount of sophisticated ranking can rescue the recommendations.

*Author's Note: Investing in reusable embeddings is a high-ROI strategic ML priority because of second-order benefits beyond accuracy improvement. Developing item embeddings in, say, a retail use case forces the product and business teams to critically examine what relevant item metadata exists, thereby enriching the catalog, cleaning the taxonomy, and landing new external data sources. Similarly, thinking about how user embeddings will be leveraged highlights data ingestion latencies that can only be solved through streaming pipelines, unblocking other teams' low-latency use cases. The embedding work becomes a forcing function for organizational data quality & infrastructure improvement.*

Near the top of the hierarchy, some organizations go as far as reusing entire models. Netflix's recent "foundation model" for recommendations attempts to train a single large

model whose embeddings and weights can be shared or fine-tuned across all recommendation surfaces. This mirrors the trajectory of NLP, where one large language model now performs tasks that once required many specialized models. Not every company has the resources to pursue this approach of a single unified model. At the very apex sits a different kind of reuse: subscribing to a managed recommendation API such as Amazon Personalize, Google Recommendations AI, or Vertex AI Search, and consuming someone else's recommender end to end rather than any component of your own. This is the most heavily packaged option on the hierarchy, since the vendor owns the data pipeline, the model, the retraining schedule, and the serving infrastructure, and it is also the least differentiating, because the resulting experience reflects the vendor's design choices rather than anything proprietary to the business. For an organization large enough to need the rest of this book, that trade is usually the wrong one, and the apex is correspondingly the narrowest and least occupied tier in the figure. But even medium-sized enterprises can benefit from the lower levels of the hierarchy, particularly the embedding layer.

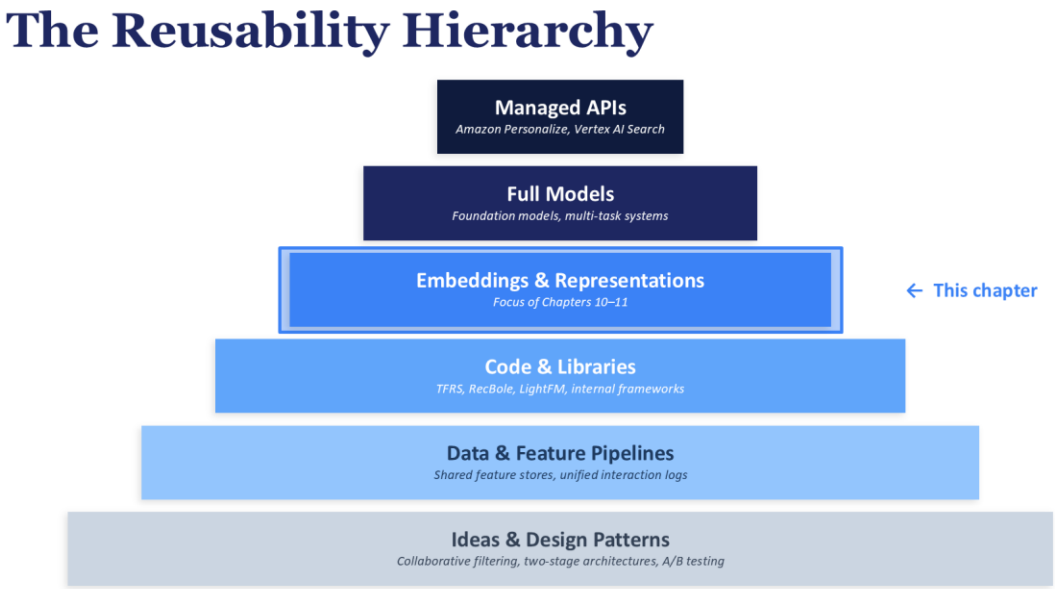



*FIGURE 1: The Reusability Hierarchy. Each tier is a type of artifact an organization can share across its recommendation surfaces, rising from ideas and design patterns at the base, through data and feature pipelines, code and libraries, embeddings and representations, and full models, to a managed recommendation API at the apex. The tiers narrow toward the top because the artifacts become rarer and more heavily packaged, not because they matter more. Embeddings and representations, the focus of Chapters 10 and 11, are highlighted.*

## Why Item Embeddings Are the Starting Point

This chapter and the next focus on the embeddings layer of the reusability hierarchy, because it offers the best balance of impact and feasibility for most organizations. Reusing ideas is valuable but abstract; reusing entire models requires enormous scale. Embeddings sit in the sweet spot: they are concrete artifacts that can be computed once and consumed

by many downstream systems, yet they do not require the organizational coordination of a full foundation model effort.

An item embedding is a dense numeric vector (typically 64 to 768 dimensions) that encodes the essential characteristics of a product, article, song, or any other entity in a recommendation catalog. Two items with similar embeddings should be similar in some meaningful sense: they might share a category, attract the same audience, or serve as substitutes for each other. This simple idea that mapping items into a vector space where proximity reflects relatedness, is what enables the efficient nearest-neighbor retrieval that powers modern recommendation systems at scale.

If the organization is just starting out in building reusable ML assets, then begin with item embeddings rather than user embeddings because items are the more stable entity + far more intuitive to be justified in front of non-ML stakeholders. For example in a retail setting, a product catalog changes slowly relative to user metadata. New items are of course added , old ones are retired and existing items have metadata changes. But the bulk of the catalog remains relatively unchanged on a daily basis. Whereas user behavior is constantly evolving and often requires more advanced modelling/infra requirement. Item embeddings can be computed in a batch process and cached for far longer (relative to user embeddings) without too much loss of freshness. They **can** be used as the input to baseline approaches to user embedding construction, as we will see in Chapter 11, a user embedding is typically some function of the item embeddings corresponding to the user's interaction history. Getting the item layer right can often be a prerequisite for everything that follows.

## Chapter Roadmap

This chapter explores item embedding construction through a progression of four approaches, each building on the previous. Section 10.4 begins with zero-shot encoding using pre-trained sentence transformers. These models require no task-specific training and provide surprisingly strong baselines that cover the entire item catalog. Section 10.5 introduces Item2Vec, which applies Word2Vec to user interaction sessions to learn purely behavioral embeddings that capture co-occurrence patterns invisible to text-based approaches. Even though the core concept for Word2Vec is over a decade old, the implementations have stood the test of time and can serve as a very powerful baseline that is hard to beat if you are just starting out. Section 10.6 combines both signals through contrastive fine-tuning which adapts pre-trained encoders with behavioral supervision. Finally, Section 10.7 explores simple approaches to multi-modal content fusion, combining embeddings from different modalities (audio, text, genre).

In the companion exercises to the book across these sections, we work with four datasets spanning news (MIND), e-commerce (Amazon KDD Cup 2023), music (Music4all-Onion and Yandex Yambda), and sponsored search (IJCAI-18 CVR). This breadth is intentional to give the readers an intuition for a few of the common dataset patterns that they might encounter in real life and to reveal how domain-specific characteristics (text richness, session length, modality availability) influence which approach works best.

It is worth noting that reusability in recommender systems extends far beyond embeddings. Shared data pipelines, common evaluation frameworks, standardized model architectures, and even full foundation models are all valuable forms of reuse. We do not attempt to cover all of these in this chapter, since that would be a book in itself. Instead, we focus on the layer that, in the author's experience, offers the most immediate and compounding returns: the learned representations that sit at the heart of every personalization system. Additionally, the item embeddings is also the layer where the overlap with Information Retrieval/Search is most apparent.

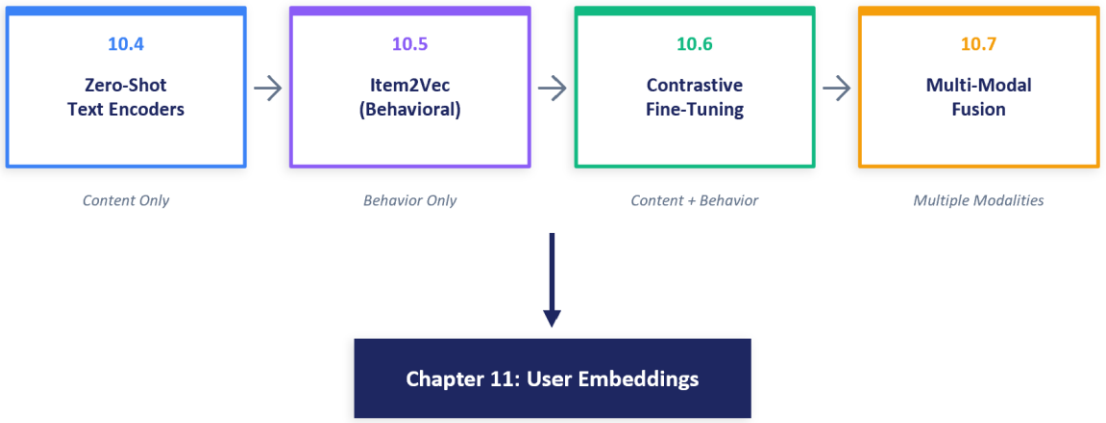



*FIGURE 2: Chapter 10 roadmap diagram showing the four exercise sections and how they build on each other, with arrows indicating progressive complexity from zero-shot to multi-modal fusion, and a note showing that Chapter 11 (User Embeddings) consumes these as inputs.*

## 10.2 Understanding "Items" Across Domains

Before diving into specific embedding methods, it is worth stepping back to consider what we mean by an "item." The techniques in this chapter apply across a wide range of industries, but the nature of the items being recommended, and the data available to describe them, varies considerably from one domain to another. Understanding these differences is important because they influence which embedding approaches are most appropriate and what evaluation strategies make sense.

In retail e-commerce platforms like Amazon or Alibaba's marketplaces, an item is a physical product often identified by a SKU or ASIN ID. These come with rich structured data: titles, descriptions, category taxonomies, brands, prices, and technical specifications. They typically have one or more product images, and sometimes videos. User interaction data is both plentiful due to the large user base and rich in granularity because of product experience maturity: views, clicks, add-to-cart events, purchases, reviews, and ratings exist for more than a decade+. Alibaba's Taobao team found that incorporating side information such as category and brand into item embeddings significantly improved recommendations, especially for new or rarely-clicked items.

In food delivery platforms like Uber Eats, the recommended "item" might be a restaurant storefront or an individual menu item ("Spinach dumpling". These items have features like cuisine type, location, average rating, price range, and delivery time estimates. Importantly, they also carry contextual relevance that shifts throughout the day such as a coffee shop is more relevant at 8 AM, a pizza restaurant at 10 PM. Embeddings for food delivery items must capture geographic relevance, cuisine similarity, and quality signals simultaneously, making this a particularly multi-faceted representation challenge.

In news and content platforms like those powered by Microsoft's MIND dataset, the items are articles, each defined primarily by text: a title, an abstract, and category labels. The lifespan of relevance is short, because a breaking news article may be relevant for hours, not weeks. User interactions (clicks, read time) provide behavioral signal, but the cold-start problem is particularly acute because new articles are published continuously and must be embedded before any user has seen them.

In music streaming, the items are songs or tracks. Each has audio characteristics (tempo, key, instrumentation), textual metadata (title, artist, genre tags, lyrics), and rich behavioral data (listening sessions, skip rates, playlist co-occurrence). Music recommendation is distinctive in its repetitiveness unlike many other domains: users replay favorite songs regularly; and also in the role of context: what a user wants to hear depends on their mood, activity, and time of day. The Music4all-Onion dataset used in Section 10.7 provides pre-extracted features across audio, lyrics, and genre modalities for this domain to give the readers a flavor of what is possibly available if you had access to a production music dataset.

In professional networks like LinkedIn, the recommended items might be job postings. These are heavily textual (job title, description, requirements) with structured attributes (industry, seniority, location). Behavioral signals include clicks, saves, and applications. The embedding challenge is semantic: "data scientist" and "ML engineer" may describe overlapping roles despite distinct titles, while "data scientist" in healthcare versus finance may require entirely different skill sets.

## Common Modalities Across Domains

Despite these differences, a recurring set of data modalities appears across nearly every recommendation domain. Recognizing these common modalities is important because each can be transformed into an embedding, and the techniques for doing so are largely transferable across industries.

Textual data is perhaps the most universal modality. Almost all domains have items that have some associated text such as product titles, article headlines, song lyrics, job descriptions. Text is a rich source of semantic information and can be transformed into embeddings using natural language processing techniques ranging from simple TF-IDF vectors to transformer-based language models like SBERT (which we explore in Section 10.4). The quality of text-based embeddings depends heavily on how descriptive and

discriminative the available text is: a detailed product description yields far better embeddings than a cryptic item ID.

Images and visual data are crucial for many product types such as apparel, furniture, food, and social media content. Pre-trained computer vision models (such as ResNet, Vision Transformers, or CLIP) can extract visual embeddings that capture style, color, and object characteristics. Pinterest's work on PinSage demonstrated the power of visual embeddings at web scale, learning graph-convolutional representations over billions of product images.

Categorical metadata provides high-level semantic grouping: category hierarchies, brands, tags, and labels. These can be encoded as one-hot vectors, learned embeddings, or included as part of a text serialization template (as we demonstrate in Section 10.4). Numeric attributes like price, rating, or popularity count can be normalized and included alongside other features, though they are typically less central to embedding construction than text or behavioral signals.

User interaction data (click logs, purchase histories, co-view and co-click relationships, session sequences) is arguably the most informative signal for item embeddings that are used in personalization. Two items frequently viewed or purchased by the same users likely share some form of similarity, even if their descriptions differ completely. This behavioral signal forms the basis of collaborative filtering approaches like Item2Vec (Section 10.5) and graph-based methods like those used by Alibaba and Pinterest.

The key insight for practitioners is that robust item embeddings typically combine multiple modalities rather than relying on any single one. Content-based signals (text, images) handle cold-start items well but miss behavioral patterns. Collaborative signals (co-occurrence, graph structure) capture user intent but fail for new items. The progression of this chapter, from text-only (Section 10.4) to behavioral (Section 10.5) to combined (Section 10.6) and multi-modal (Section 10.7), mirrors the path many production systems follow as they mature.

## Common Modalities Across Domains

| Modality | Retail E-commerce | Food Delivery | News / Media | Music Streaming | Professional / Jobs |
|---|---|---|---|---|---|
| Text | ✓ | ✓ | ✓ | ✓ | ✓ |
| Images | ✓ | ✓ | ✓– | ✓ | – |
| Categorical Metadata | ✓ | ✓ | ✓ | ✓ | ✓ |
| Numeric Attributes | ✓ | ✓ | – | ✓ | ✓ |
| Behavioral (clickstream) | ✓ | ✓ | ✓ | ✓ | ✓ |

*FIGURE 3: Table showing the common modalities (Text, Images, Categorical, Numeric, Behavioral) and which are available in each domain (Retail, Food Delivery, News, Music, Jobs), with checkmarks indicating availability.*

## 10.3 What Makes a Good Item Embedding?

Before diving into specific methods, it is worth establishing what we are optimizing for. A good item embedding should satisfy several properties that enable it to serve multiple downstream use cases.

### Semantic Coherence

Items that humans would consider similar should have nearby embeddings. A blue fleece blanket should be close to a grey sherpa throw compared to a pair of jeans; a news article about the NBA playoffs should be close to other sports articles rather than articles about politics. This is the most intuitive property and can be verified qualitatively by inspecting nearest neighbors in the embedding space. If you query for the nearest neighbors of "wireless headphones" and get back items like "kitchen knife set" in the top few results, the embeddings are not semantically coherent (or rarely but more alarmingly the associated product metadata is seriously low quality).

### Behavioral Alignment

Items that users treat as related by co-viewing, co-purchasing, or sequential browsing should also be nearby in latent space. This is subtly different from semantic coherence. A phone case and a screen protector are semantically dissimilar (different categories, different descriptions) but behaviorally co-engaged. Item embeddings trained on the appropriate data can capture these complementary relationships that pure content-based approaches would miss.

### Discriminative Power

Embeddings must also be discriminative such that distinct items should have distinct representations. An embedding space where every news article about politics maps to nearly the same vector is semantically coherent but useless for retrieval, because we cannot distinguish between articles within the same topic. This tension between semantic grouping and individual discrimination is a recurring theme throughout the chapter.

### Efficiency and Reusability

Finally, embeddings should be compact enough for efficient nearest-neighbor search and reusable across multiple downstream tasks without re-computation. A 384-dimensional embedding can be indexed in FAISS or similar libraries for fast nearest neighbor retrieval across millions or billions of items. Once computed, these embeddings can be stored and consumed by retrieval, ranking, and personalization models alike. In the Appendix on Hybrid Search with Vector Databases, we contrast these dense vectors with bag-of-words and learned sparse models, whose vocabulary-sized sparse vectors are served from an inverted index.

### Evaluation Strategy

Throughout this chapter, we evaluate embeddings using retrieval-based metrics that directly test whether the embedding space is useful for recommendation. The primary evaluation paradigm is next-item retrieval: given the embedding of the last item a user interacted with, can we retrieve the actual next item from the full catalog? We measure this with:

- **Precision@K:** The fraction of the top-K retrieved items that are relevant.
- **Recall@K:** The fraction of all relevant items that appear in the top-K.
- **NDCG@K:** A ranking-aware metric that gives more credit when relevant items appear higher in the list.
- **MRR (Mean Reciprocal Rank):** The average of 1/rank for the first relevant item. If the true next item appears at position 3, its reciprocal rank is 1/3.

We additionally can perform qualitative evaluation by inspecting nearest neighbors such as by examining whether the closest items in embedding space share brands, categories, or topical themes. This provides an intuitive sanity check that complements the quantitative metrics across the entire set.

## 10.4 Pre-trained Text Encoder Baselines (Zero-Shot)

The simplest way to generate item embeddings is to encode each item's textual metadata using a pre-trained language model. No task-specific training is required since the model is used as-is, downloaded straight from Hugging Face or wherever its hosted. This approach is fast to deploy and provides a strong baseline that is often competitive with more complex methods.

### From Word Embeddings to Sentence Transformers: A Brief History

To appreciate why sentence transformers work so well for item embedding, it helps to understand the lineage of ideas that led to them. The story begins with Word2Vec (Mikolov et al., 2013), which showed that training a simple neural network to predict words from their surrounding context produces word embeddings where semantic relationships are encoded as vector arithmetic. The famous example of "king" minus "man" plus "woman" approximately equals "queen" demonstrated that these embeddings capture meaning in a geometrically interpretable way. We will use this same Word2Vec architecture for item embeddings in Section 10.5 (Item2Vec).

The original Word2Vec operates at the word level, producing one embedding per word. To represent a full sentence or paragraph (such as a product description), early approaches simply averaged the word embeddings: a "bag of words" strategy that discards word order and grammatical structure. This works reasonably well for short texts but loses important nuances: "a dog bites a man" and "a man bites a dog" have identical average embeddings despite very different meanings.

BERT (Devlin et al., 2018) addressed this by training a deep transformer network on a masked language modeling objective: given a sentence with some words replaced by

[MASK] tokens, the model learns to predict the missing words. This forces the model to develop rich contextual representations where the embedding of a word depends on the words around it, not just the word itself. BERT's pre-trained representations dramatically advanced the state of the art in NLP. However, BERT was not designed to produce fixed-length **sentence** embeddings suitable for similarity search between documents which have more than a few words. Extracting a useful sentence embedding from BERT requires task-specific fine-tuning, and using BERT to compare all pairs in a large catalog is prohibitively expensive (comparing 10,000 sentences requires approximately 50 million inference passes).

Sentence-BERT (Reimers and Gurevych, 2019) addressed both of these problems. The core issue with using BERT directly for embeddings is that BERT was not trained to produce useful fixed-length sentence representations. When practitioners extract a sentence embedding from BERT, typically by pooling (averaging) the token-level outputs or taking the [CLS] token's hidden state, the resulting vector is not optimized for semantic similarity. Studies have shown that these naive pooling strategies can produce embeddings where cosine similarity does not correspond to semantic relatedness, sometimes performing no better than averaging static GloVe word vectors. The embedding space is not structured for comparison because BERT was trained for token-level prediction (masked language modeling), not for producing globally meaningful sentence representations.

SBERT solves this by fine-tuning BERT in a Siamese network configuration: two copies of the same BERT model process two sentences independently, and the network is trained with a contrastive or regression objective on labeled sentence pairs. This training explicitly optimizes the embedding space so that semantically similar sentences produce nearby vectors. As a secondary benefit, the Siamese architecture also dramatically improves efficiency: because each 'sentence' is encoded independently into a fixed vector, comparing N sentences requires only N forward passes (linear cost) rather than the $N^2/2$ cross-encoding passes that raw BERT would need. For item embedding, this means we can encode a catalog of hundreds of thousands of items in minutes rather than days.

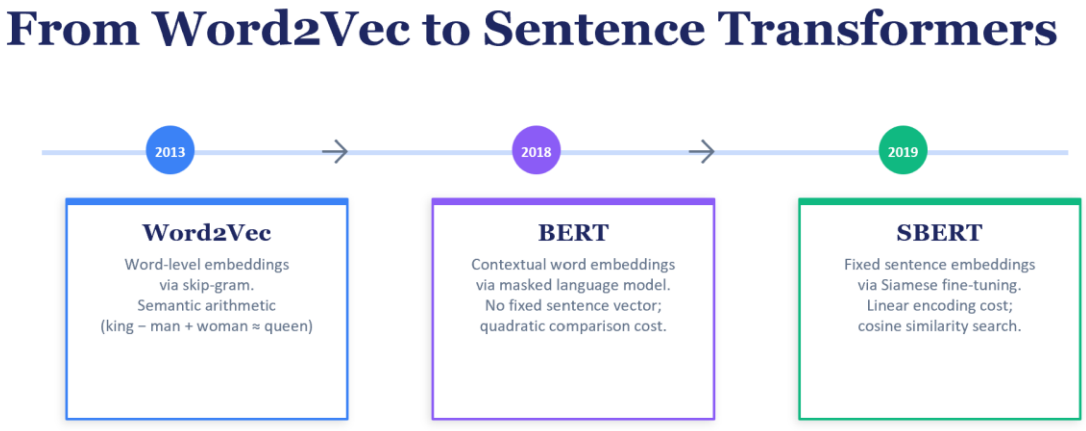



*FIGURE 4: Evolution from Word2Vec (2013, word-level skip-gram) to BERT (2018, contextual masked language modeling) to SBERT (2019, Siamese fine-tuning for fixed-length sentence*

*embeddings). Each step addresses a limitation of the previous one, culminating in a model that produces fixed-length, semantically meaningful embeddings with linear encoding cost.*

## A Closer Look at all-MiniLM-L6-v2

The specific model we use most frequently in our exercises is all-MiniLM-L6-v2, a widely adopted sentence transformer that has become a default first choice for embedding-based retrieval across industry and research. Understanding its architecture and training helps explain why it works so well as an item encoder.

MiniLM-L6-v2 is a 6-layer transformer (the "L6" in the name) with a hidden dimension of 384 and 12 attention heads. It is a distilled model, meaning it was trained to mimic the behavior of a larger teacher model (in this case, a 12-layer model) while using fewer parameters. The distillation process, introduced by the MiniLM paper (Wang et al., 2020), transfers knowledge from teacher to student by training the student to reproduce the teacher's attention distributions and value-relation matrices. The result is a model that achieves approximately 95% of the teacher's quality at roughly half the computational cost.

The "all" prefix indicates that the model was further fine-tuned on a large collection of sentence pair datasets spanning diverse domains such as natural language inference, semantic textual similarity, question-answer pairs, and paraphrase detection. This broad fine-tuning gives the model a general understanding of semantic similarity that transfers well to new domains. When we encode product titles or news article headlines with this model, it leverages this general understanding even though it was never trained on e-commerce or news data.

The 384-dimensional output is produced by mean-pooling the last hidden layer's token embeddings: averaging across all token positions to produce a single fixed-length vector. This is simpler than using the [CLS] token (which BERT was designed around) but has been shown to produce better sentence embeddings in practice. The output vectors are typically L2-normalized before similarity computation, ensuring that cosine similarity and dot product become equivalent—an important property for efficient FAISS indexing.

For ML engineers evaluating sentence transformer models, some of the key trade-offs are: output dimensionality (384 vs. 768 vs. 1024), model size (number of layers and parameters, which affects inference latency), context window (eg. 512 tokens) and the breadth of the training data (which determines how well the model generalizes to new domains). MiniLM-L6-v2 sits at an attractive point in this space: compact enough for production use, yet expressive enough to produce high-quality embeddings across a wide range of text types. It is a sensible default for any team beginning to explore embedding-based retrieval. Its also extensively benchmarked & documented in various technical blog posts and tutorials.

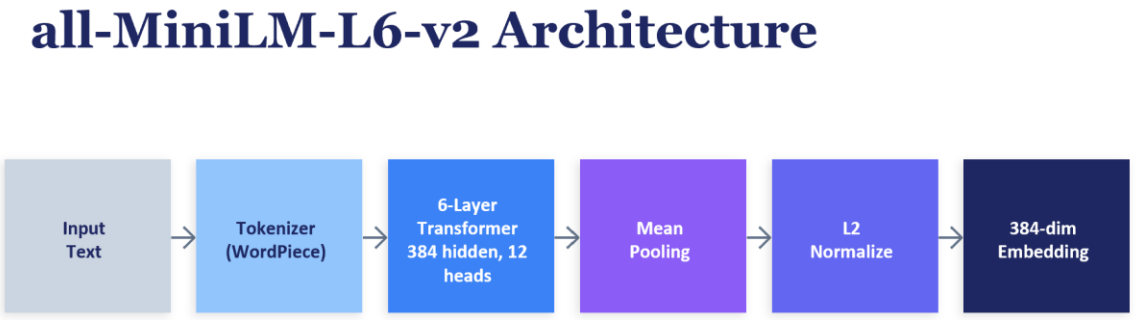



*FIGURE 5: all-MiniLM-L6-v2 architecture pipeline: input text → WordPiece tokenizer → 6-layer transformer (384 hidden dimensions, 12 attention heads) → mean pooling over token embeddings → L2 normalization → 384-dimensional output embedding. The model is distilled from a larger 12-layer teacher model, achieving roughly 95% of the teacher's quality at roughly half the computational cost.*

## Sentence Transformers (SBERT)

Sentence Transformers, introduced by Reimers and Gurevych (2019), are BERT-based models that have been fine-tuned specifically to produce semantically meaningful sentence embeddings. Unlike raw BERT outputs (which require task-specific fine-tuning to be useful), SBERT models are trained with a contrastive or regression objective on sentence pairs, producing embeddings where cosine similarity directly corresponds to semantic similarity.

For item embedding, we serialize each item's metadata into a text string and feed it through the SBERT encoder. The model outputs a fixed-dimensional vector (typically 384 or 768 dimensions) that captures the semantic content of the item. For example, a product with the title "SOCHOW Sherpa Fleece Throw Blanket" and brand "SOCHOW" might be serialized as:

```
SOCHOW Sherpa Fleece Throw Blanket [SEP] SOCHOW [SEP] Super soft plush blanket...
```

The choice of serialization template matters. For the Amazon KDD dataset, we concatenate the product title, brand, and description with [SEP] tokens as delimiters. For the MIND news dataset, we combine the article title, abstract, and category. The template should include the most discriminative textual fields available.

## Model Comparison: General-Purpose vs. Domain-Specialized

In the companion exercises to this chapter, we compare two categories of pre-trained encoders:

- General-purpose SBERT models like all-MiniLM-L6-v2 (384-dim) and all-mpnet-base-v2 (768-dim) have been trained on a broad corpus of natural language inference and semantic similarity tasks. They understand language at a general level—synonyms, paraphrases, topical relatedness but have no specific knowledge of e-commerce catalogs or user behavior.

- Domain-specialized models like RexBERT, which was pre-trained with masked language modeling (MLM) on about 2.3 trillion tokens, including about 350 billion tokens of e-commerce text, have deep knowledge of product terminology and catalog structure. However, RexBERT lacks a contrastive training objective because it was trained to predict masked tokens, not to produce embeddings where similar products are nearby.

This distinction is critical. RexBERT's MLM training teaches it to understand product language (it knows that "fleece" and "sherpa" are related fabric types), but the resulting embeddings are not optimized for similarity search. SBERT's contrastive training, by contrast, directly optimizes for the property we need: similar inputs produce nearby embeddings. In our exercises, general-purpose SBERT outperforms domain-specialized RexBERT on retrieval tasks on the Amazon KDD dataset, despite having no e-commerce domain knowledge. While further validation across additional datasets would strengthen this conclusion, the result suggests that the training objective matters more than domain coverage for embedding quality.

## Evaluation on MIND (News Domain)

The MIND dataset provides approximately 50,000 news articles with rich textual metadata: titles, abstracts, categories, and entity annotations. We evaluate zero-shot SBERT embeddings through two lenses:

We evaluate zero-shot SBERT embeddings on MIND through two retrieval tasks. The first, category retrieval, tests whether the nearest neighbors of each article share its news category (sports, politics, technology, etc.). The following code snippet illustrates the core evaluation pattern—encoding items, building a FAISS index, and measuring whether nearest neighbors are topically coherent:

```
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

# 1. Encode all items into embeddings
model = SentenceTransformer("all-MiniLM-L6-v2")
texts = [f"{title} [SEP] {abstract} [SEP] {category}" for ...]
embeddings = model.encode(texts, normalize_embeddings=True)

# 2. Build FAISS index for fast nearest-neighbor search
dim = embeddings.shape[1]  # 384
index = faiss.IndexFlatIP(dim)  # Inner product = cosine for normalized vectors
index.add(embeddings)

# 3. For each article, retrieve top-K neighbors and check category match
```

```
distances, indices = index.search(embeddings, k=11)  # k+1 to exclude self
for i, neighbors in enumerate(indices):
    neighbors = neighbors[1:]  # Remove self-match at position 0
    same_category = sum(categories[n] == categories[i] for n in neighbors)
```

This pattern—encode, index, search, evaluate—recurs throughout the chapter. The FAISS IndexFlatIP index performs exact inner-product search; for L2-normalized embeddings, inner product is equivalent to cosine similarity. On the MIND dataset, SBERT achieves strong category coherence among nearest neighbors, confirming that the pre-trained model captures topical similarity effectively even without any domain-specific training.

The second evaluation, co-click retrieval, tests whether articles clicked by the same user (anywhere in that user's click history) end up nearby in embedding space. Here, the results are far more modest. Co-click reflects behavioral similarity such as two articles a user reads in succession, but this may not align with textual similarity (depending on the domain). A user might read a sports article and then a celebrity gossip article because both appeared on the homepage, not because they are topically related. This gap between content similarity and behavioral similarity in some domains is a key motivation for the behavioral embedding approaches in later sections. (In other domains like e-commerce shopping, it's more likely that customers are viewing similar items in the same session rather than completely different product categories.)

## Evaluation on Amazon KDD (E-Commerce Domain)

The Amazon KDD Cup 2023 dataset provides approximately 500,000 English products and 1.18 million shopping sessions. We evaluate using next-item retrieval: given the last product a user viewed in a session, can we retrieve the actual next product from the full 500K catalog?

For the Amazon KDD evaluation, we construct a next-item retrieval task from session data. Each session provides a sequence of products a user browsed; we use the last item as a query and check whether the actual next product appears among the nearest neighbors in the full catalog. This evaluation directly tests whether the embedding space supports the kind of item-to-item similarity that a retrieval system would exploit:

```
# Build ground truth: (query_item, target_item) from sessions
pairs = []
for session in sessions:
    prev_items = session["prev_items"]  # Items browsed so far
    next_item = session["next_item"]     # The actual next engagement
    query_id = prev_items[-1]            # Last viewed item = query
    pairs.append((query_id, next_item))


# Encode full catalog and build FAISS index
```

```
item_embeddings = encoder.encode_dict(item_texts)  # {id: 384-dim vector}
index = build_faiss_index(embeddings_matrix)


# For each query, search for nearest items and check if target is retrieved
for query_id, target_id in pairs:
    query_emb = item_embeddings[query_id]
    distances, neighbor_ids = index.search(query_emb, k=50)
    hit = target_id in neighbor_ids
```

With no training at all, a general-purpose sentence transformer retrieves the exact next product as one of the closest items in a catalog of hundreds of thousands of products for a meaningful fraction of sessions. This establishes a strong baseline that more complex methods must beat to justify their additional cost.

## When Zero-Shot Fails: Tokenization and the IJCAI Homework

Not all catalogs have rich textual metadata, and understanding why zero-shot encoding fails on certain data requires understanding how text encoders process their input. At the heart of every transformer model is a tokenizer which is a preprocessing step that converts raw text into a sequence of integer token IDs (from its pretrained vocabulary) that the model can process.

Modern tokenizers (like WordPiece used by BERT and SBERT) do not operate on whole words. Instead, they maintain a vocabulary of subword units, common words are represented as single tokens, while rare or unknown words are broken into smaller pieces. For example, the word “unforgettable” might be tokenized as [“un”, “##forget”, “##table”], where “##” indicates a continuation of the previous token. This subword strategy allows the model to handle any input text, even words it has never seen during training, by decomposing them into familiar fragments.

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")


# Meaningful product title: subwords carry semantic information
tokens = tokenizer.tokenize("Wireless Noise-Cancelling Headphones")
# -> ["wireless", "noise", "-", "cancel", "##ling", "headphones"]


# Hashed ID: subwords are arbitrary digit fragments
tokens = tokenizer.tokenize("8734290157823461029")
# -> ["87", "##34", "##29", "##01", "##578", "##23", "##46", "##10", "##29"]
```

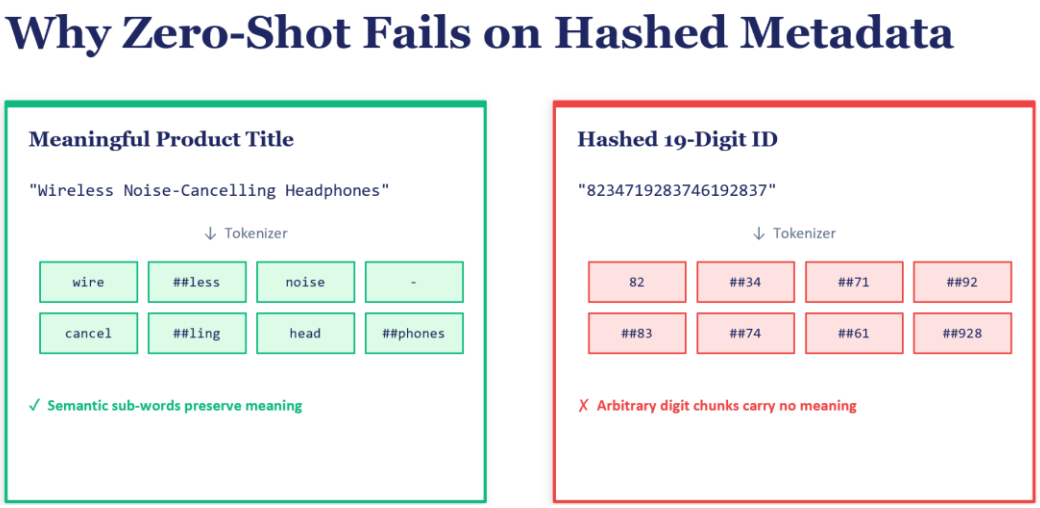



*FIGURE 6: Side-by-side comparison showing tokenization of a meaningful product title vs. a hashed 19-digit ID, illustrating how subword tokens carry semantic meaning for natural text but become arbitrary digit fragments for numeric IDs.*

This is precisely what happens with the IJCAI-18 sponsored search dataset from Alibaba. Category and property identifiers have been replaced with hashed 19-digit integers for anonymization. When the tokenizer processes these numeric IDs, it breaks them into arbitrary sequences of digit fragments that carry no semantic meaning—the token “87” followed by “##34” means nothing to the model. As a result, all items produce nearly identical embeddings (the model cannot distinguish between them), and retrieval performance drops to near zero.

This is a practical lesson for ML engineers: zero-shot text encoding works well when items have meaningful natural language descriptions, but fails when the metadata is anonymized, hashed, or purely numeric. For such domains, the behavioral approaches in Sections 10.5 and 10.6 become essential, as they learn similarity from user actions rather than from item descriptions.

## Text Serialization for Structured Metadata

Most item catalogs contain structured metadata which are not usually in the format that non-GenAI small language models used for item embeddings expect. In the ecommerce, a product might have a brand field, a category hierarchy, a price bucket, and a set of attribute tags, none of which are natural sentences. To use text encoders on such data, we serialize the structure into pseudo-natural-language text:

```
Category: clothing > women > dresses | Properties: cotton, summer | Brand: Zara |
Price: medium
```

This template-based serialization is a practical technique that bridges the gap between structured catalogs and text-based encoders. The key is to include field names (like "Category:" and "Brand:") so the encoder can use them as context clues. While not as effective as training on native structured data, this approach enables immediate use of powerful pre-trained language models on any catalog.

## 10.5 Item2Vec: Collaborative Embeddings from Sequences

Pre-trained text encoders capture content similarity. The core idea being items with similar descriptions end up with similar embeddings. But personalization is fundamentally about behavioral similarity: which items do users treat as related through their revealed preferences? A phone case and a screen protector have very different descriptions, but users who buy one frequently buy the other. Content-based embeddings miss this signal entirely.

### Sidebar: Revealed Preferences in Recommender Systems

The distinction between content-based and behavioral embeddings connects to a foundational concept in economics and behavioral science: revealed preferences. The theory, introduced by economist Paul Samuelson in 1938, holds that a consumer's true preferences are best understood not by what they say they like, but by what they actually choose when presented with options. In the context of recommendation, this means that a user's click, browse, and purchase history which is their revealed behavior is often a more reliable signal of their preferences than any stated profile or survey response.

Author's note: This is also the reason why companies can't rely on hand-labelling golden datasets at **scale** for creating supervised training dataset for ranking problems. Aside from the obvious issues of not having enough coverage even with an army of human labelers, their results will be too biased based on whatever the specific human labelers want rather than being representative of the preferences across millions of customers. This nuance about revealed preferences also tends to be a stumbling block for new teams trying to convince leadership why user feedback is needed at all! (In the author's experience, the debate is usually settled only when the supervised models trained on user feedback data finally goes into production and the accuracy improvements are too enormous to be dismissed)

Content-based embeddings capture what items are (their attributes, descriptions, categories). Behavioral embeddings capture how users relate to items through their actions, their revealed preferences. When a user browses a phone case immediately after viewing a smartphone, the browsing sequence reveals that these items are related in the user's mind, regardless of how different their textual descriptions may be. Item2Vec and the contrastive fine-tuning in Section 10.6 both learn from these revealed preference signals, encoding them into the embedding space.

This perspective also explains why behavioral embeddings tend to outperform content embeddings for frequently interacted items: the revealed preference data directly encodes the similarity judgments that matter for recommendation, while content features are only a proxy. Conversely, for cold-start items with no interaction data, content embeddings are the only option—we must fall back on what the item is rather than how users have treated it.

Item2Vec (Barkan and Koenigstein, 2016) addresses this gap by applying the Word2Vec algorithm to user interaction sequences. The key insight is a direct analogy: words that appear in similar contexts tend to have similar meanings (the distributional hypothesis in linguistics), and items that appear in similar user sessions tend to serve similar user needs. By treating each user session as a "sentence" and each item as a "word," we can train a skip-gram model to learn item embeddings that reflect co-occurrence patterns.

## The Word2Vec Foundation

Item2Vec's effectiveness rests on the Word2Vec algorithm (Mikolov et al., 2013), one of the most influential ideas in representation learning. Word2Vec trains a shallow neural network on a simple task: given a word in a sentence, predict the words that surround it (the "skip-gram" variant) or, conversely, given the surrounding words, predict the center word (the "CBOW" variant). The remarkable discovery was that the hidden layer weights, once trained on large text corpora, encode rich semantic relationships between words. Words that appear in similar linguistic contexts end up with similar embedding vectors.

The skip-gram variant, which we use for Item2Vec, works as follows. For each word in the training corpus, a context window of fixed size defines the neighboring words. The model learns to predict these context words given the center word, using a single hidden layer. Training with negative sampling (Skip Gram Negative Sampling - SGNS) makes this efficient: rather than computing a full softmax over the entire vocabulary for each training example, the model samples a small number of random "negative" words and learns to distinguish the true context words from these negatives. This reduces the per-example computation from O(vocabulary size) to O(number of negatives), making training feasible on corpora with millions of words or, in our case, millions of items.

The analogy to recommendation is direct. In Word2Vec, words that appear in similar sentences develop similar embeddings because they are predicted by similar context words. In Item2Vec, items that appear in similar user sessions develop similar embeddings because they co-occur with similar other items. A phone case and a screen protector may have completely different text descriptions, but if they consistently appear together in shopping sessions, their Item2Vec embeddings will converge. This purely behavioral signal is complementary to the content-based signal captured by the text encoders in the previous section.

## How Item2Vec Works

The training process follows the Word2Vec skip-gram architecture with negative sampling:

1. Construct item sequences from user behavior. Each user session (a sequence of browsed or purchased items) becomes a training sentence. This is where the ML team needs to spend the most time to make sure that the dataset has high signal to noise ratio (for example relatively free of bots, etc.)
2. For each item in a sequence, define a context window of neighboring items. The skip-gram objective trains the model to predict context items given the center item.

3. Use negative sampling for efficiency: for each positive (center, context) pair, sample several random items as negatives. The loss pushes the center item's embedding closer to context items and away from random items.
4. After training, each item has a learned embedding vector (typically 128–256 dimensions) that captures its position in the behavioral space.

The following code snippet shows the key design choices in our Item2Vec implementation. Note how the minimum count threshold is derived from the data rather than set arbitrarily:

```
from gensim.models import Word2Vec
from collections import Counter
import numpy as np


# sessions: List[List[str]], each session is a sequence of item IDs


# Derive min_count from the item frequency distribution
freq = Counter(item for session in sessions for item in session)
counts = np.array(list(freq.values()))
min_count = max(int(np.percentile(counts, 5.0)), 1)  # 5th percentile


# Train Skip-gram with negative sampling
model = Word2Vec(
    sentences=sessions,    # Each session is a "sentence"
    vector_size=128,       # Embedding dimensionality
    window=5,              # Context window size
    min_count=min_count,   # Data-driven threshold
    sg=1,                  # Skip-gram (not CBOW)
    negative=10,           # Negative samples per positive
    epochs=10,
    workers=4,
)


# Access embeddings: model.wv["item_id"] -> numpy array
# Find similar: model.wv.most_similar("item_id", topn=10)
```

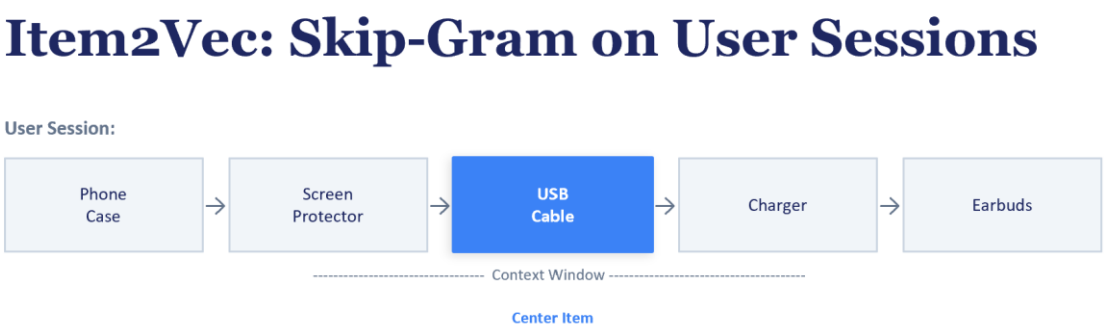



*FIGURE 7: Item2Vec skip-gram training. A user session is treated as a sequence of items, and a sliding context window centers on each item in turn. For each (center_item, context_item) positive pair, several random items from the catalog are drawn as negative samples. The model learns to push the center item's embedding toward context items and away from random negatives, encoding co-occurrence patterns into the embedding space.*

## Key Design Choices

Minimum count threshold: Items that appear in very few sessions do not have enough co-occurrence data to learn meaningful embeddings. We set a minimum count threshold based on the item frequency distribution. Items below this threshold are excluded from training. This is a crucial practical decision: including extremely rare items would produce noisy embeddings that pollute the vector space without adding value.

Coverage vs. quality trade-off: Item2Vec can only produce embeddings for items that meet the minimum count threshold. On the Amazon KDD dataset, this means coverage is limited to frequently appearing items: a significant fraction of the 500K catalog may be excluded. Text-based encoders, by contrast, cover 100% of items because they only need metadata. This coverage gap is a fundamental limitation of purely behavioral approaches and motivates the hybrid strategies explored in later sections.

## Content vs. Behavioral Embeddings: A Comparison

Comparing Item2Vec with zero-shot SBERT on the Amazon KDD dataset reveals a complementary pattern. When we restrict evaluation to the subset of items covered by both methods (an apples-to-apples comparison), Item2Vec tends to excel: behavioral co-occurrence directly encodes what users treat as substitutes or complements. However, on the full catalog, text-based embeddings dominate because they cover the entire long tail of items that lack behavioral signal.

This complementarity is one of the most important lessons in item embedding design: content and behavioral signals capture fundamentally different aspects of item similarity. Content tells us what an item is; behavior tells us how users relate to it. The best production systems leverage both.

### Item2Vec on Music Sessions (Yandex Yambda)

To demonstrate the generality of the Item2Vec approach, we also train it on the Yandex Yambda music listening dataset, which contains over 50 million listening events. Music sessions exhibit different properties than shopping sessions, they are typically longer, more repetitive (users replay favorite songs), and driven by mood and genre rather than purchase intent. Item2Vec captures these listening patterns, learning that songs frequently heard in the same session (or by the same user) likely share genre, tempo, or mood characteristics.

## 10.6 Contrastive Fine-Tuning

Sections 10.4 and 10.5 established two complementary approaches: content-based embeddings that cover all items but miss behavioral patterns, and collaborative embeddings that capture user behavior but only cover frequently appearing items. Contrastive fine-tuning bridges this gap by starting with a pre-trained text encoder and adapting it using behavioral supervision.

The idea is straightforward: take the SBERT model that already understands product language and fine-tune it so that items which users engage with sequentially end up closer in the embedding space. This preserves SBERT's content understanding (it still knows that "fleece blanket" and "sherpa throw" are similar) while adding a behavioral signal (phone case moves closer to screen protector because users frequently browse them in sequence).

### Multiple Negatives Ranking Loss (MNR)

The training uses Multiple Negatives Ranking Loss (MNR), a contrastive loss that leverages in-batch negatives for efficiency. For each training batch of size B, we have B positive pairs (anchor item, positive item). The loss treats every other item in the batch as a negative, yielding B × (B−1) negative comparisons per batch—for free, without explicit negative sampling.

Concretely, the loss computes the cosine similarity between every anchor and every positive in the batch, producing a B × B similarity matrix. The diagonal entries (correct pairs) should be high; off-diagonal entries (incorrect pairs) should be low. A cross-entropy loss over each row, with the diagonal as the target, achieves this. The similarity matrix is scaled by a temperature parameter (typically 20, equivalent to temperature=0.05) to sharpen the softmax distribution and produce stronger gradient signals.

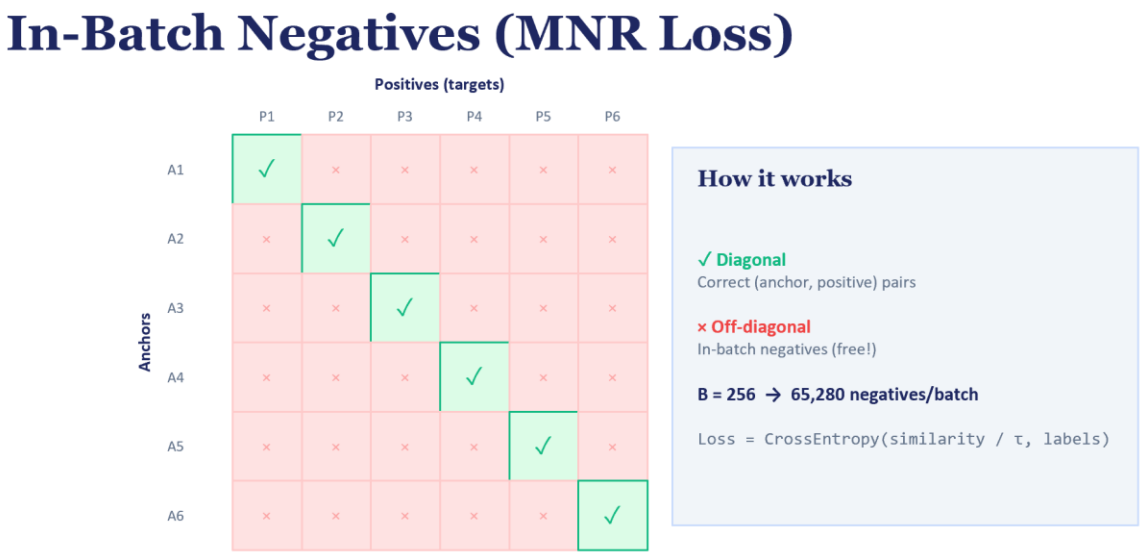



*FIGURE 8: Diagram showing in-batch negative construction: a B×B similarity matrix with the diagonal representing positive pairs and off-diagonal entries representing negatives.*

The implementation is concise. The core loss function computes the full similarity matrix and applies cross-entropy with the diagonal as targets:

```
def mnr_loss(anchor_embs, positive_embs):
    """In-batch negatives: B anchors x B positives -> B*B similarity matrix."""
    # Cosine similarity between all pairs: (B, B) matrix
    similarity = F.cosine_similarity(
        anchor_embs.unsqueeze(1),    # (B, 1, D)
        positive_embs.unsqueeze(0),  # (1, B, D)
        dim=-1
    )
    # Scale by temperature (20.0 = 1/0.05)
    similarity = similarity * 20.0
    # Diagonal = correct pairs, off-diagonal = negatives
    labels = torch.arange(len(similarity), device=similarity.device)
    return F.cross_entropy(similarity, labels)
```

## Constructing Positive Pairs

The choice of positive pairs determines what the fine-tuned embeddings learn to capture. In a real use case , this will require extensive discussions with the domain experts/analytics/business teams. For our worked examples, we make the following assumptions:

- For Amazon KDD, we construct (last-viewed item, next-engaged item) pairs from session data. If a user's session shows they viewed item A and then engaged with item B,

the pair (A, B) becomes a training example. This teaches the model that sequential browsing indicates relatedness.

- For MIND, we construct co-click pairs: if a user clicked both article A and article B (anywhere in that user's click history), the pair (A, B) is a positive example. This teaches the model to place articles read by the same person near each other in embedding space, even when their topics differ.

### Results and Analysis

Contrastive fine-tuning produces consistent improvements over zero-shot baselines across all metrics. The gains are most pronounced at higher recall thresholds meaning the fine-tuned model pushes relevant items significantly closer to the query item, even if the top-1 result doesn't always change. The best checkpoint is typically found after just one epoch of training, with subsequent epochs showing mild overfitting.

The improvements are modest rather than dramatic, and there is an important pedagogical reason for this. The zero-shot baseline is already quite strong: SBERT's general text understanding captures product similarity well from descriptions alone. The behavioral pairs encode browsing patterns (phone case → screen protector) rather than deep semantic similarity, and each item appears in only a handful of training pairs on average, limiting the fine-tuning signal. In production systems with orders of magnitude more interaction data, contrastive fine-tuning typically yields larger gains.

Recall the earlier finding that domain-specialized RexBERT (trained with MLM on e-commerce text) underperformed general-purpose SBERT. Contrastive fine-tuning explains why: RexBERT has domain knowledge but lacks a contrastive objective. Section 10.6 demonstrates what happens when we add that objective. Even with modest behavioral data, the contrastive signal improves retrieval quality beyond what domain-specific language modeling alone can achieve.

## 10.7 Multi-Modal Content Fusion

Many items are described by more than just text. A song has audio characteristics (tempo, instruments, mood), textual information (lyrics, genre tags), and metadata (artist, album, release year). A product has images, descriptions, and structured attributes. In theory, combining these modalities should produce richer embeddings than any single modality alone. In practice, as we will discover, naive fusion can actually degrade performance.

We explore multi-modal fusion using the Music4all-Onion dataset, which provides pre-extracted features for approximately 109,000 tracks across three modalities:

- Audio i-vectors (100 dimensions): Compact representations of acoustic characteristics.
- Lyrics Word2Vec embeddings (300 dimensions): Semantic representations of lyrical content.
- Genre TF-IDF vectors (685 dimensions): Sparse representations of genre tags.

## Approach A: PCA Baseline (Concatenate and Reduce)

The simplest fusion strategy concatenates all modality vectors into a single 1,085-dimensional vector and applies PCA to reduce it to 128 dimensions. This is a non-parametric approach because no learning is involved beyond finding the principal components. The appeal is simplicity: we get a single compact vector that contains information from all three modalities.

The problem is that PCA finds the directions of maximum variance in the concatenated space, which may not correspond to the directions most useful for recommendation. If genre features have high variance (685 dimensions of sparse binary vectors), PCA may allocate most of its 128 output dimensions to capturing genre variation, effectively drowning out audio and lyrics information or conversely, it might dilute the genre signal that was most useful.

## Approach B: CLIP-Style Contrastive Alignment

Inspired by OpenAI's CLIP model (Radford et al., 2021), we train projection heads that align audio and lyrics features in a shared 128-dimensional space. For each track, the (audio, lyrics) pair is a positive example; other tracks in the batch serve as negatives. The InfoNCE contrastive loss pushes corresponding audio and lyrics representations together while separating non-corresponding pairs.

This approach learns real correspondences between modalities. The training loss decreases well below the random baseline, confirming that the model learns to associate audio characteristics with lyrical themes. However, learning audio–lyrics correspondence is not the same as learning what predicts co-listening, a distinction that becomes apparent in the evaluation.

## How CLIP-Style Training Works

The CLIP (Contrastive Language–Image Pre-training) framework, introduced by Radford et al. (2021) at OpenAI, provides a general recipe for aligning representations from two different modalities in a shared embedding space. While the original CLIP model aligned images with text captions, the same contrastive alignment principle can be applied to any pair of modalities: in our case, audio features and lyrics features for the same track.

The architecture consists of two projection heads which are small neural networks (typically two-layer MLPs) that map each modality's raw feature vector into a shared lower-dimensional space. In our implementation, the audio projection head maps 100-dimensional i-vectors to 128 dimensions, and the lyrics projection head maps 300-dimensional Word2Vec vectors to the same 128-dimensional space. Both outputs are L2-normalized so that similarity is measured by cosine distance.

The training uses an InfoNCE contrastive loss (also known as the NT-Xent loss in the SimCLR literature). For each batch of B tracks, we have B pairs of (audio_embedding, lyrics_embedding). The loss computes a B × B similarity matrix where entry (i, j) is the cosine similarity between audio_i and lyrics_j, scaled by a learnable temperature parameter.

The diagonal entries (matching pairs from the same track) should be high; off-diagonal entries (mismatched pairs from different tracks) should be low. The loss is the average of two cross-entropy terms: one treating audio as queries and lyrics as keys (each row of the similarity matrix), and one treating lyrics as queries and audio as keys (each column). This symmetric formulation ensures both modalities are equally well-aligned.

The learnable temperature parameter controls the sharpness of the softmax distribution. A low temperature (e.g., 0.07, which is CLIP's default) produces a peaked distribution that strongly penalizes confusion between the correct match and close negatives. A high temperature produces a flatter distribution with weaker gradients. In practice, the temperature is initialized to a 'reasonable' value and learned jointly with the projection head parameters.

After training, we obtain aligned embeddings for both modalities. To produce a single fused embedding per track, we average the projected audio and lyrics embeddings and re-normalize. This fused vector can then be used for retrieval in the same way as any other embedding: indexed in FAISS and queried by similarity.

The important lesson from our experiments is that CLIP-style alignment learns genuine cross-modal correspondences (the training loss decreases well below random), but learning audio–lyrics correspondence is not the same as learning what predicts co-listening. The alignment optimizes for cross-modal matching (given an audio clip, find the matching lyrics), not for the downstream task (given a track, find tracks the same user would listen to). This gap between the training objective and the evaluation task is a general challenge in representation learning, one that task-aware fusion strategies (as discussed in the Appendix on Hybrid Search with Vector Databases) aim to address.

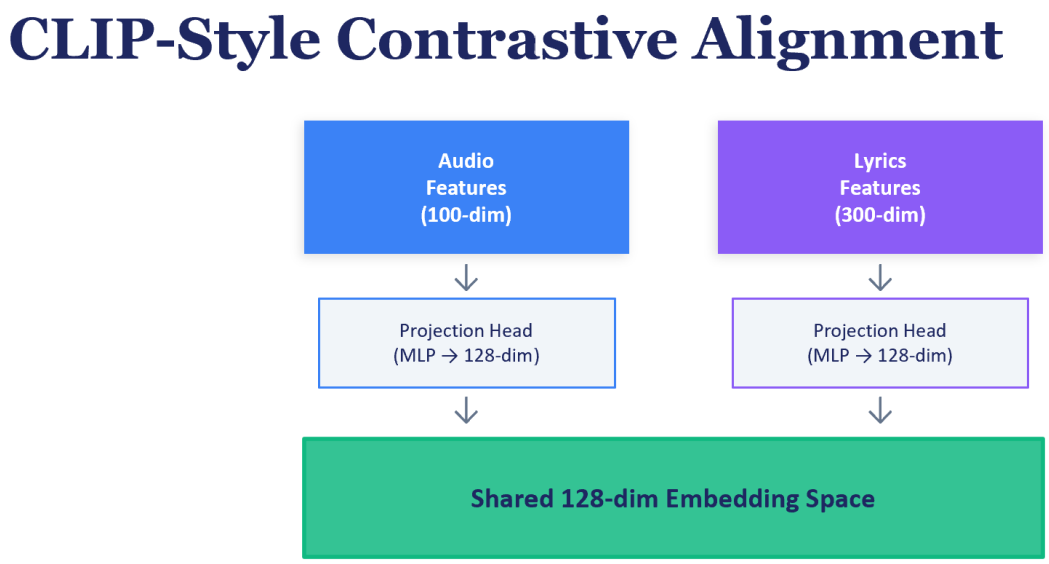



*FIGURE 9: CLIP-style contrastive alignment of two modalities. Audio features (100-dimensional) and lyrics features (300-dimensional) are each fed through a projection head (small MLP) into a shared 128-dimensional embedding space. A symmetric InfoNCE loss with*

*learnable temperature pushes matching (audio, lyrics) pairs from the same track together while pushing mismatched pairs apart. After training, the aligned representations can be averaged to produce a single fused embedding per track.*

## 10.8 The Bi-Encoder Architecture for Retrieval

Before moving to user embeddings in Chapter 11, it is worth revisiting the architecture that ties item embeddings to the retrieval stage of a recommendation pipeline. The bi-encoder (also called dual encoder or two-tower) architecture is the standard approach for embedding-based retrieval at scale.

In a bi-encoder, one encoder processes the query (a user profile, a search query, or the user history sequence) and produces a 'query' embedding. A separate encoder processes each candidate item and produces an item embedding. Relevance is measured by the similarity (typically dot product or cosine similarity) between the query and item embeddings. Because the item encoder is independent of the query, item embeddings can be pre-computed offline and indexed for fast approximate nearest-neighbor search.

The IJCAI homework exercise demonstrates a bi-encoder setup where one encoder processes user search intent (predicted category and property) and the other encodes item metadata. The dot product between these embeddings predicts conversion likelihood. This architecture decouples the query and item representations, enabling sub-millisecond retrieval across millions of items.

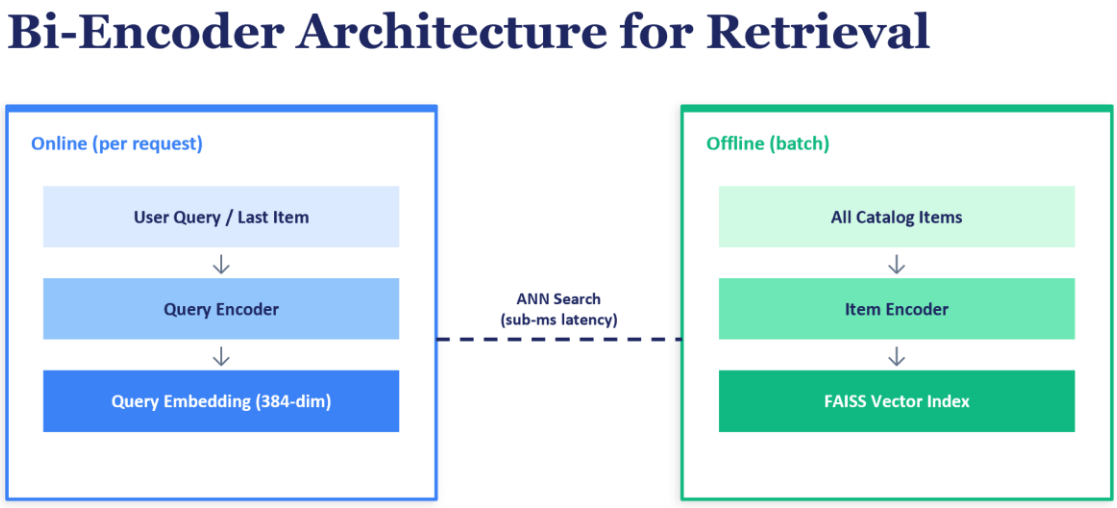



*FIGURE 10: Bi-encoder architecture diagram showing query encoder and item encoder producing embeddings in a shared space, with FAISS index for approximate nearest-neighbor retrieval.*

## 10.9 Industry Case Studies

The item embedding techniques explored in this chapter are not merely academic exercises but powerful starting point for production. Two case studies illustrate how leading companies have deployed reusable item embeddings as a platform capability.

## Uber Eats: Two-Tower Embeddings as a Platform

Uber's journey with item embeddings on Uber Eats provides a compelling illustration of the reusability thesis that opened this chapter. Initially, Uber built separate recommendation models for each city, resulting in thousands of bespoke (DeepFM) models! They replaced this with a single global Two-Tower Embedding (TTE) model that encodes both eaters (users) and restaurants (items) into a shared vector space. The architecture is straightforward: one neural network tower processes user features (order history, location, time of day) and produces a user embedding; a separate tower processes restaurant features (cuisine type, rating, delivery speed) and produces an item embedding. Relevance is measured by the dot product between these embeddings.

The critical innovation was not the model architecture itself (two-tower models were already well-established) but the decision to make it a platform. After proving the TTE model on restaurant recommendations, Uber integrated it into their ML platform so that any team could train and deploy their own two-tower model (presumably on the surface-specific clickstream events) without building infrastructure from scratch. The restaurant embeddings were reused as features in downstream ranking models, and even in fraud detection. As the Uber team noted, a single global model replaced thousands of city-specific models, and the embeddings could be scaled, reused, and transferred beyond the initial use case. This is the reusability hierarchy in action: a single embedding artifact serving multiple business needs simultaneously.

From an operational standpoint, Uber's approach decoupled the item embedding computation from real-time serving. Restaurant embeddings were pre-computed offline and indexed in an approximate nearest-neighbor service (their internal SIA system). At serving time, only the user embedding needs to be computed on the fly, enabling sub-10-millisecond retrieval across millions of restaurants. This offline-online split is the same pattern we implement in our exercises using FAISS.

## Pinterest: Multi-Modal Embeddings at Web Scale (ItemSage)

Pinterest's ItemSage system (presented at KDD 2022) demonstrates how to build a single, reusable item embedding that serves multiple recommendation surfaces simultaneously. Pinterest deals with visual content (buyable product Pins with images, titles, and descriptions) and needs embeddings that work for home feed personalization, related product recommendations, and shopping search results.

ItemSage's architecture processes each product through a multi-modal pipeline. Up to 20 product images are pre-encoded through Pinterest's PinSage visual model into feature vectors, while 12 text fields (title, brand, description, category tags) are tokenized and embedded via a learned hash embedding technique that handles a large vocabulary efficiently. These 32 feature tokens are then fed into a single-layer transformer encoder, whose [CLS] output becomes the unified product embedding.

The key design decision was multi-task training. Rather than training separate embeddings for each recommendation surface, Pinterest trains one model on multiple objectives

simultaneously: predicting user engagement on closeup views (image query to product), predicting search clicks (text query to product), and predicting purchase actions (add-to-cart, checkout). Each task provides a different supervision signal, and the shared representation must balance all of them. Pinterest reported that multi-task training actually improved performance on each individual task compared to training separately, because sparse engagement labels in one task could be supplemented by denser signals from another.

The resulting embeddings are computed in a daily batch workflow and served through an HNSW approximate nearest-neighbor index, with daily refreshes to capture new products. Different recommendation surfaces consume the same embedding. The home feed uses them for candidate retrieval, the related pins system uses them for similarity, and the search system uses them as a dense retrieval signal alongside traditional keyword matching. This is precisely the reusability pattern we advocate: invest once in high-quality embeddings, consume them everywhere.

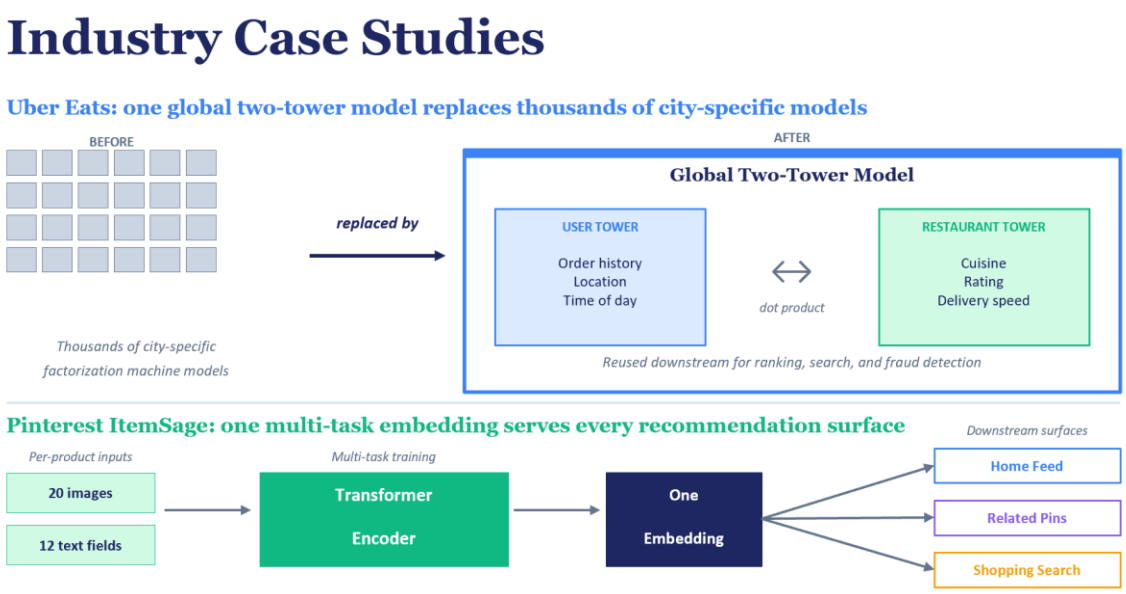



*FIGURE 11: Industry case a) Top: Uber Eats replaced thousands of city-specific models with a single global two-tower embedding model, integrated the architecture into the Michelangelo ML platform, and reused the resulting embeddings across ranking and fraud detection.*

*b) Bottom: Pinterest's ItemSage combines up to 20 product images and 12 text fields per item through a transformer encoder, trained jointly on multiple recommendation objectives (home feed, related pins, shopping search) so that a single embedding serves every surface.*

## 10.10 Managing Embeddings in Production

### Embedding Freshness and the Update Lifecycle

Item embeddings are not static artifacts. New items arrive daily (requiring embedding computation), existing items change (descriptions are updated, user behavior shifts), and

the overall catalog evolves. A production embedding system must address multiple lifecycle questions including how often to re-compute embeddings, how to handle new items between full re-computations, and how to ensure downstream consumers are not disrupted by embedding updates.

The re-computation frequency depends on the embedding type and the domain. For content-based embeddings (SBERT), new items can be embedded on arrival by running them through the encoder, no retraining required. The model is frozen; only the input changes. For behavioral embeddings (Item2Vec, contrastive fine-tuned models), re-computation requires retraining on updated interaction logs, which typically happens on a daily or weekly cadence. Some organizations maintain a two-tier system: content-based embeddings provide immediate coverage for new items, while behavioral embeddings are periodically refreshed to capture evolving user patterns.

## Version Control and Backward Compatibility

When an embedding model is retrained, the resulting embedding space may rotate, scale, or shift relative to the previous version, even if it captures the same underlying relationships. This creates a subtle but serious production problem: downstream models that consumed the old embeddings as features may behave unpredictably with the new embeddings, because the meaning of specific dimensions has changed. An item that was at position [0.3, -0.7, ...] in v1 might be at [-0.5, 0.2, ...] in v2, even though its relative similarity to other items is preserved.

Netflix addresses this through orthogonal alignment: after retraining the embedding model, they apply a low-rank orthogonal transformation that aligns the new embedding space with the old one, preserving the meaning of dimensions across versions. This allows downstream models to continue using embeddings without retraining. For organizations without the resources to implement alignment, a practical alternative is versioned embedding tables: tag each embedding batch with a version identifier (e.g., v1, v2) and retrain downstream consumers whenever the embedding version changes. This is somewhat disruptive and requires coordination between ML teams owning the embeddings, the data engineering/Feature Store infra team hosting the embeddings and the downstream consumer of the embeddings. But its conceptually easier to communicate across the org and simpler to implement.

## Storage and Serving Infrastructure

In production, item embeddings live in two places: an offline feature store (for batch consumption by training pipelines) and an online vector index (for real-time retrieval). The feature store should support versioning, point-in-time lookups (to avoid training-serving skew), and efficient bulk reads. The vector index, typically FAISS, ScaNN, Milvus, or a managed service like Pinecone or Weaviate, must support fast approximate nearest-neighbor queries at low latency (10s of milliseconds for millions of items).

The embedding dimensionality directly affects both storage cost and retrieval speed. For very large catalogs, product quantization or binary embeddings (as used by Tencent's BEBR

system) can compress embeddings further with minimal quality loss. As a practical guideline, start with 384 dimensions and only increase if there is evidence of quality improvement on your specific task. There are some advanced techniques like Matroyshka Representation Learning (MRL) that allows minimal information loss when the embeddings are truncated. Readers are encouraged to explore the rich literature around this topic.

### Onboarding Teams Onto a Shared Embedding Platform

The organizational challenge of reusable embeddings is as significant as the technical one. Getting multiple teams to adopt a shared embedding rather than training their own requires trust, documentation, and demonstrated value. Uber's approach of integrating two-tower training into their ML platform (so teams could customize their own tower architectures while sharing the infrastructure) is a proven pattern. At a minimum, a shared embedding service should provide: well-documented APIs for embedding lookup by item ID, clear versioning so teams know when embeddings change, quality dashboards showing nearest-neighbor coherence and retrieval metrics, and an embedding explorer tool that lets teams qualitatively inspect neighbors for items in their domain. Without these supporting artifacts, even high-quality embeddings will struggle to gain adoption across an organization.

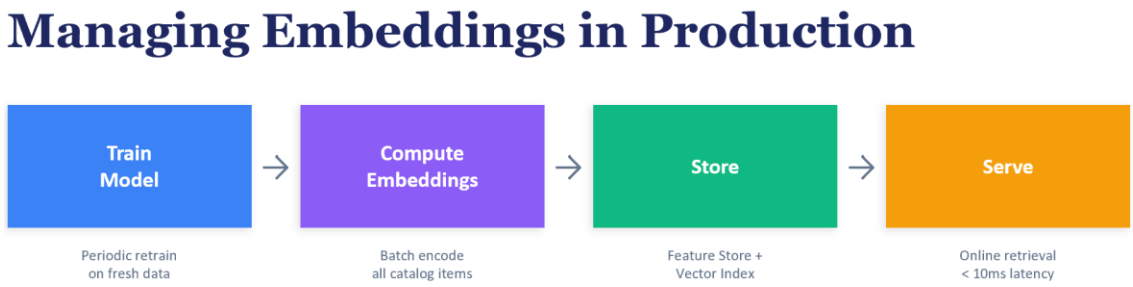



*FIGURE 12: Embedding lifecycle in production: Train Model → Compute Embeddings → Store (feature store + vector index) → Serve (online retrieval). Offline batch jobs refresh the model and re-encode the catalog periodically; online requests look up pre-computed embeddings and query the vector index for sub-10ms retrieval. Key production concerns at each stage: version control across retrains (orthogonal alignment), cold-start handling for new items, freshness SLOs, and onboarding of downstream consumer teams.*

## 10.11 Connecting to User Embeddings

This chapter established that item embedding quality is the foundational layer upon which all downstream personalization is built. In Chapter 11, we take these item embeddings and ask: given a user's interaction history (a sequence of items they have engaged with), how do we construct a representation of the user?

The simplest approach of averaging the embeddings of items a user has interacted with already leverages everything we built in this chapter. A user who browses phone cases, screen protectors, and USB cables will have their average embedding in the "mobile accessories" region of the space, naturally retrieving related items. But as we will see, more

sophisticated approaches (learned sequence models, graph neural networks) can improve upon this baseline by learning which items in a user's history are most predictive of their next action.

A critical finding that carries over from this chapter: in the simplistic way of creating user embedding above, its usefulness is often bounded by item embedding quality. No matter how sophisticated the user model, if the underlying item embeddings do not capture meaningful similarity, the user embeddings will inherit those limitations. Investing in better item embeddings, as we did by progressing from zero-shot to fine-tuned models, pays dividends across the entire recommendation stack.

In the subsequent chapter, we will cover two popular approaches to jointly learning user & item embeddings represented by the earlier case studies. The first is the two-tower model (as demonstrated by Uber in Section 10.9), where separate encoder networks for users and items are trained simultaneously with a contrastive or classification loss that pushes relevant user–item pairs together. The second is graph neural networks (such as LightGCN), which learn user and item embeddings jointly through message passing on the user–item interaction graph. Both approaches produce user and item embeddings in the same vector space, enabling direct similarity-based retrieval.

## 10.12 Summary

This chapter explored four approaches to item embedding construction, each revealing a different aspect of what makes items similar:

- **Pre-trained text encoders** provide strong zero-shot baselines that cover the entire catalog. They capture semantic similarity from descriptions but miss behavioral patterns. The training objective (contrastive vs. MLM) matters more than domain specialization.
- **Item2Vec** learns purely behavioral embeddings from user sessions, capturing co-occurrence patterns that content analysis misses. However, it only covers items with sufficient interaction frequency, leaving the long tail unrepresented.
- **Contrastive fine-tuning** combines the strengths of both approaches by adapting pre-trained encoders with behavioral supervision. The gains are consistent but modest with limited data; production systems with more interaction data typically see larger improvements.
- **Multi-modal fusion** taught us that more modalities do not automatically mean better embeddings. When one modality dominates the prediction task, naive fusion dilutes it. The correct approach is task-aware weighted combination with per-modality normalization.

The overarching lesson is that item embeddings are not a one-size-fits-all artifact. The right approach depends on your data regime: rich text metadata favors content-based methods; dense interaction logs favor collaborative methods; and mature production systems benefit from combining both. Regardless of the method chosen, the investment in item embedding quality pays compounding returns across retrieval, ranking, and personalization.

# References


5. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP.
6. Barkan, O. and Koenigstein, N. (2016). Item2Vec: Neural Item Embedding for Collaborative Filtering. IEEE MLSP Workshop.
7. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., and Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. Advances in NeurIPS.
8. Radford, A., Kim, J.W., Hallacy, C., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML.
9. Henderson, M., Al-Rfou, R., Strope, B., et al. (2017). Efficient Natural Language Response Suggestion for Smart Reply. arXiv:1705.00652.
10. Karpukhin, V., Oguz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP.
11. Wu, F., Qiao, Y., Chen, J.-H., et al. (2020). MIND: A Large-scale Dataset for News Recommendation. Proceedings of ACL.
12. Santana, I., Gomes, M.A., Calumby, R.T., et al. (2020). Music4All-Onion — A Large-Scale Multi-faceted Content-Centric Music Recommendation Dataset. ACM ICMR.
13. Covington, P., Adams, J., and Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. Proceedings of RecSys.
14. Wang, J., Huang, P., Zhao, H., et al. (2018). Billion-scale Commodity Embedding for E-commerce Recommendation in Alibaba. KDD.
15. Ying, R., He, R., Chen, K., et al. (2018). Graph Convolutional Neural Networks for Web-Scale Recommender Systems (PinSage). KDD.
16. Zhai, J., Liao, L., Liu, X., et al. (2024). Actions Speak Louder Than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations (HSTU, Meta). ICML.
17. Yi, X., Yang, J., Hong, L., et al. (2019). Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations (Google Two-Tower). RecSys.
18. Sculley, D., Holt, G., Golovin, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in NeurIPS. https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf
19. Netflix Technology Blog (2024). Foundation Model for Personalized Recommendation. https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39
20. Wang, W., Wei, F., Dong, L., et al. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. NeurIPS.
21. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
22. Samuelson, P. A. (1938). A Note on the Pure Theory of Consumer's Behaviour. Economica, 5(17), 61–71.

23. Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). ICML.